1. Database technology used

Component	Technology
Backend	Python
API framework	FastAPI
Database	SQLite
Production database option	MySQL 8.0+
Database file	`job_portal.db`
Database setup	`setup_database.py`
Table definitions	`schema_sqlite.sql`
Sample records	`seed_data.sql`
SQL reports	`queries_and_reports.sql`

Your application automatically creates the SQLite database when you run `python run.py` and the database file does not exist.

2. All 17 database tables

These are the actual tables defined in your repository.

No.	Table name	Purpose
1	`users`	Registration and login
2	`companies`	Company information
3	`job_seekers`	Job seeker profiles
4	`recruiters`	Recruiter profiles
5	`subscription_plans`	Free, Premium and Enterprise plans
6	`recruiter_subscriptions`	Recruiter subscription details
7	`payments`	Payment transactions
8	`invoices`	Payment invoices and GST
9	`jobs`	Job postings
10	`skills`	Available skills
11	`job_skills`	Skills required for jobs
12	`seeker_skills`	Skills possessed by candidates
13	`resumes`	Uploaded resumes
14	`applications`	Job applications
15	`application_tracking`	Application status history
16	`saved_jobs`	Bookmarked jobs
17	`notifications`	User notifications

3. Table columns and primary keys

Below are the main columns in each table, based on `schema_sqlite.sql`.

A. User and profile tables

• users

Column	Data type	Key
user_id	INTEGER	PK
full_name	TEXT	
email	TEXT	UNIQUE
password_hash	TEXT	

Column	Data type	Key
phone	TEXT	
role	TEXT	
is_active	INTEGER	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

• companies

Column	Data type	Key
company_id	INTEGER	PK
company_name	TEXT	UNIQUE
industry	TEXT	
website	TEXT	
location	TEXT	
company_size	TEXT	
description	TEXT	
logo_url	TEXT	
is_verified	INTEGER	
created_at	TIMESTAMP	

• job_seekers

`seeker_id` (PK), `user_id` (FK), `headline`, `qualification`, `experience_years`, `current_location`, `expected_salary`, `bio`, `linkedin_url`, `github_url`, `created_at`, `updated_at`.

• recruiters

`recruiter_id` (PK), `user_id` (FK), `company_id` (FK), `designation`, `department`, `is_verified`, `created_at`, `updated_at`.

B. Job management tables

• jobs

`job_id` (PK), `recruiter_id` (FK), `company_id` (FK), `job_title`, `job_type`, `experience_level`, `location`, `salary_min`, `salary_max`, `description`, `requirements`, `open_vacancies`, `is_featured`, `deadline`, `status`, `created_at`, `updated_at`.

• skills

`skill_id` (PK), `skill_name`, `category`.

• job_skills

`job_id` (PK, FK), `skill_id` (PK, FK), `is_required`.

• seeker_skills

`seeker_id` (PK, FK), `skill_id` (PK, FK), `proficiency_level`.

C. Application and resume tables

• resumes

`resume_id` (PK), `seeker_id` (FK), `file_name`, `file_path`, `file_size_kb`, `is_primary`, `uploaded_at`.

• applications

`application_id` (PK), `job_id` (FK), `seeker_id` (FK), `resume_id` (FK), `cover_letter`, `applied_at`, `current_status`, `updated_at`.

• application_tracking

`tracking_id` (PK), `application_id` (FK), `status`, `remarks`, `changed_by_user_id` (FK), `changed_at`.

• saved_jobs

saved_id (PK), seeker_id (FK), job_id (FK), saved_at.

D. Subscription and payment tables

• subscription_plans

plan_id (PK), plan_name, price, duration_days, job_posting_limit, candidate_search_limit, featured_jobs_allowed, priority_support, created_at.

• recruiter_subscriptions

subscription_id (PK), recruiter_id (FK), plan_id (FK), start_date, end_date, status, auto_renew, created_at.

• payments

payment_id (PK), subscription_id (FK), amount, payment_method, transaction_id, payment_status, payment_date, gateway_response.

• invoices

invoice_id (PK), payment_id (FK), invoice_number, tax_amount, total_amount, issued_date, invoice_pdf_path.

E. Notification table

• notifications

notification_id (PK), user_id (FK), title, message, notification_type, is_read, created_at.

4. Sample records available in your repository

The seed_data.sql file contains the following preloaded demo records.

Table	Sample records
users	11
companies	5
job_seekers	6
recruiters	4
subscription_plans	3
recruiter_subscriptions	4
payments	3
invoices	3
jobs	6
skills	12
job_skills	17
seeker_skills	19
resumes	6
applications	8
application_tracking	16
saved_jobs	5
notifications	6
Total	128

These are the sample INSERT records in the repository, not a live count of your current localhost database. Your application may have additional records created through registration, job posting, applications and other activities.

Examples of actual sample records

Users table

user_id	full_name	role
1	System Administrator	admin
2	Rajesh Varma	recruiter

user_id	full_name	role
3	Priya Nair	recruiter
6	Kamal Sen	job_seeker
7	Ananya Patel	job_seeker

Companies table

company_id	company_name	Industry
1	Infosys Technologies	Information Technology
2	Tata Consultancy Services	Enterprise Software
3	FinEdge Innovations	Financial Technology
4	CloudScale Systems	Cloud & DevOps
5	NextGen AI Labs	Artificial Intelligence

Jobs table

job_id	Job title	Location
1	Full Stack Python Developer	Bengaluru
2	Senior Cloud DevOps Architect	Bengaluru
3	React Frontend Engineer	Pune
4	Java & Spring Boot Engineer	Mumbai
5	FinTech Data & SQL Analyst	Hyderabad
6	Junior Backend Developer (Python)	Remote

Subscription plans table

plan_id	Plan name	Price
1	Free Starter	■0
2	Recruiter Premium	■499
3	Enterprise Unlimited	■999

5. All four database views

Your database also defines four views for dashboard reporting.

View name	Purpose
`vw_active_jobs_with_company`	Active jobs with company details
`vw_recruiter_dashboard_stats`	Recruiter performance and job statistics
`vw_job_seeker_applications`	Candidate application dashboard
`vw_admin_revenue_and_subscriptions`	Admin revenue and subscription analytics

Example:

```
SELECT *
FROM vw_active_jobs_with_company;

SELECT *
FROM vw_recruiter_dashboard_stats;

SELECT *
FROM vw_job_seeker_applications;

SELECT *
FROM vw_admin_revenue_and_subscriptions;
```

6. SQL queries used in your project

Your repository includes a dedicated `queries_and_reports.sql` file containing queries for the different modules.

Here are the important queries.

Query 1: Display all users

```
SELECT * FROM users;
```

Query 2: Display all job seekers

```
SELECT *
FROM users
WHERE role = 'job_seeker';
```

Query 3: Display all recruiters with company names

```
SELECT
    r.recruiter_id,
    u.full_name,
    u.email,
    c.company_name
FROM recruiters r
JOIN users u
    ON r.user_id = u.user_id
JOIN companies c
    ON r.company_id = c.company_id;
```

Query 4: Display all available jobs

```
SELECT
    j.job_id,
    j.job_title,
    c.company_name,
    j.location,
    j.salary_min,
    j.salary_max
FROM jobs j
JOIN companies c
    ON j.company_id = c.company_id
WHERE j.status = 'open';
```

Query 5: Search jobs by location

```
SELECT *
FROM jobs
WHERE location LIKE '%Hyderabad%';
```

Query 6: Find jobs with salary above ■5 lakhs

```
SELECT *
FROM jobs
WHERE salary_min >= 500000;
```

Query 7: Display job applications with candidate names

```
SELECT
    a.application_id,
    u.full_name,
    j.job_title,
    a.current_status
FROM applications a
JOIN job_seekers js
    ON a.seeker_id = js.seeker_id
JOIN users u
    ON js.user_id = u.user_id
JOIN jobs j
    ON a.job_id = j.job_id;
```

Query 8: Find shortlisted candidates

```
SELECT *
FROM applications
WHERE current_status = 'shortlisted';
```

Query 9: Display payment history

```
SELECT
    payment_id,
    amount,
    payment_method,
    transaction_id,
    payment_status
FROM payments;
```

Query 10: Calculate total revenue

```
SELECT
    SUM(total_amount) AS total_revenue
FROM invoices;
```

Query 11: Count jobs posted by each recruiter

```
SELECT
    recruiter_id,
    COUNT(job_id) AS total_jobs
FROM jobs
GROUP BY recruiter_id;
```

Query 12: Display unread notifications

```
SELECT *
FROM notifications
WHERE is_read = 0;
```

Query 13: Find the most popular skills

```
SELECT
    s.skill_name,
    COUNT(js.job_id) AS demand_count
FROM skills s
JOIN job_skills js
    ON s.skill_id = js.skill_id
GROUP BY s.skill_id, s.skill_name
ORDER BY demand_count DESC;
```

Query 14: Display recruiter subscription details

```
SELECT
    r.recruiter_id,
    u.full_name,
    sp.plan_name,
    sp.price,
    rs.start_date,
    rs.end_date,
    rs.status
FROM recruiters r
JOIN users u
    ON r.user_id = u.user_id
JOIN recruiter_subscriptions rs
    ON r.recruiter_id = rs.recruiter_id
JOIN subscription_plans sp
    ON rs.plan_id = sp.plan_id;
```

Query 15: Find the highest-paying jobs

```
SELECT
    job_title,
    salary_max
FROM jobs
ORDER BY salary_max DESC
LIMIT 5;
```

7. Advanced SQL concepts used

Your project demonstrates these database concepts:

SQL concept	Usage
CREATE	Creating database tables
INSERT	Adding users, jobs and records
SELECT	Retrieving information
UPDATE	Updating application status
DELETE	Removing records
WHERE	Filtering records
ORDER BY	Sorting jobs and salaries
GROUP BY	Grouping recruiters and skills
HAVING	Filtering grouped results
INNER JOIN	Combining related tables
LEFT JOIN	Including unmatched records
Subqueries	Admin dashboard KPIs
Aggregate functions	COUNT, SUM, AVG
CASE	Application status conditions

SQL concept	Usage
COALESCE	Handling NULL values
Window functions	DENSE_RANK
Views	Dashboard reports
Triggers	Automatic application tracking
Stored procedures	Transaction-based job applications

8. Primary key and foreign key relationships

These are the important relationships used in your ER diagram.

Parent table	Child table	Relationship
users	job_seekers	One-to-One
users	recruiters	One-to-One
companies	recruiters	One-to-Many
companies	jobs	One-to-Many
recruiters	jobs	One-to-Many
job_seekers	resumes	One-to-Many
job_seekers	applications	One-to-Many
jobs	applications	One-to-Many
applications	application_tracking	One-to-Many
skills	job_skills	One-to-Many
skills	seeker_skills	One-to-Many
recruiters	recruiter_subscriptions	One-to-Many
subscription_plans	recruiter_subscriptions	One-to-Many
recruiter_subscriptions	payments	One-to-Many
payments	invoices	One-to-One
users	notifications	One-to-Many

9. Triggers and stored procedures

Your project includes additional database automation.

Trigger: `trg_after_application_status_update`

When an application's status changes, the SQLite trigger:

• Inserts a record into `application_tracking`.

• Creates a notification for the job seeker.

MySQL stored procedure: `sp_apply_for_job`

This procedure handles job application processing with transaction-related validation.